A practical dynamic-form pilot

Replace one email-and-spreadsheet request journey with a configurable form inside the existing React product.

Recommendation: use the Form.io React wrapper behind a small integration component. Keep the existing product API responsible for identity, permissions, request data and workflow.

Business outcome

Requesters provide the right information for equipment or software access, save drafts, and see progress in one place. Reviewers receive consistent data and an auditable status history. The initial scope is one process with two request types.

How value will be measured

During the pilot, measure completed requests, requests returned for missing information, and time from submission to first review. Establish the email/spreadsheet baseline with the process owner. No quantified savings are claimed before those measurements.

Evidence and scope

Delivered in the prototype	Proposed for customer integration
React + JSON renderer, two conditional panels, client/API validation	Reuse real product authentication, database and scoped permissions
Draft/final submission, persisted reload, role behavior, EN/VI labels	Real file upload, scan, controlled download and retention
Version selection, activity history, error recovery and tests	Existing workflow handoff and controlled form publication

Assessment boundary: synthetic users/data, local API, metadata-only attachments and seeded approval statuses. Customer UAT is planned, not completed.

Scope that users can understand

A representative journey

1. Choose a request type. **2.** Complete the relevant section. **3.** Save an incomplete draft or submit a validated request. **4.** Reopen the request and track its status. A submitted request is read-only under the proposed policy.

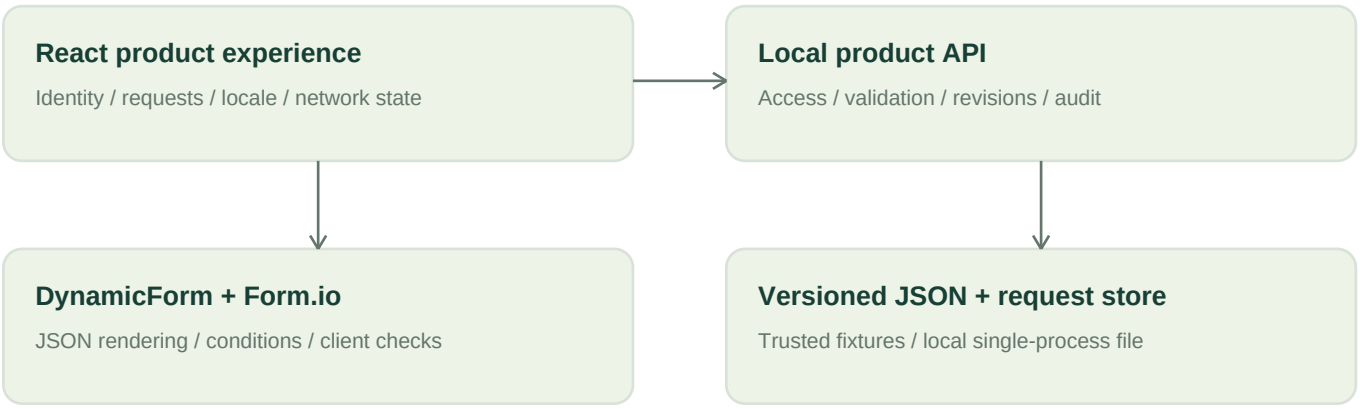
Concern	Working behavior / explicit boundary
Request types	Equipment: item + quantity. Software: application + access duration. Changing type discards inactive answers on save.
Who can do what	Requester: create and edit own drafts, read own records. Reviewer: read all records. Role/ownership checks run in the API under simulated identity.
Validation	Final submission requires title, reason, type and active-section fields. Drafts may omit required fields but supplied values must satisfy type/bounds.
Files, status and audit	File metadata only. In review / Approved are seeded states. Server writes actor/time/action on draft saves and submissions.
Languages	English/Vietnamese app copy and form labels/options. Some API diagnostics and renderer hints remain English.

Discovery decisions before a real pilot

Confirm the field set, access scope, approval/resubmission rules, attachment policy, translation owner, configuration author/approver and named UAT owner. Default field limits are assessment assumptions, not customer-confirmed business policy.

Excluded: a custom builder or form engine, renderer migration, new workflow platform, microservices, production SSO implementation and arbitrary untrusted schema uploads.

Clear integration boundaries



The browser fetches a JSON definition from the local API and supplies it to the Form.io wrapper. For existing requests, it first reads the record, then fetches that record's pinned version and loads its submission data.

What each layer owns

React: identity context, selected request, language, loading/error state and navigation. **Form.io:** live field values, conditional visibility and client validation. **API:** authorization, domain validation, status, revision and persistence.

Live form values stay in the renderer; a ref reads them on save. React uses local state for the product shell. There is no duplicated global field store or neutral schema conversion layer.

Submission and failure behavior

Final submit validates locally, then sends a stable request ID, formVersion, revision, data and action. The API checks policy, strips inactive values, validates and writes the record with audit before acknowledging success. A 503 preserves input. A stale revision returns 409 and requires reload.

Product integration: replace the mock API boundary with the existing verified session, entities, file service and workflow. Keep actor/status authoritative on the server. Hiding a control is not authorization.

The local JSON store is single-process and unsuitable for shared production writes. Use the existing database and workflow dispatch facilities; this prototype does not implement distributed delivery guarantees.

Configuration with controlled change

A small, demonstrable versioning rule

New requests load the active version. Saved requests keep their original version. In the demo, changing active.json from 1 to 2 adds the pre-supported Additional notes field to new requests without rebuilding the frontend; existing v1 records still load v1.

Important boundary: JSON can change supported presentation and field visibility. A new persisted field or business rule may need a backend contract change. Configuration does not remove domain compatibility work.

Proposed publication flow

Author -> preview with sample data -> compatibility and permission checks -> business approval -> publish immutable version -> activate for new requests. Retain old versions. Rollback changes the active pointer; migration of saved records requires a separate explicit decision.

Security and operational ownership

Only trusted authors may publish. Allow approved component types and endpoints; reject unreviewed script rules. Derive real identity from the product session and scope server queries. Reuse approved file scanning/download/retention controls. Avoid payload contents in operational logs.

Dependency trade-off

The wrapper contains renderer lifecycle and locale integration. It does not remove Form.io schema/rule coupling. Pin releases, run representative form regressions before upgrades, monitor advisories and verify required component support. The installed packages declare MIT; no Enterprise capability or hosted Form.io project is used.

Measured build: approximately 2.09 MB minified JavaScript / 610 KB gzip. Route-level lazy loading and target-device performance checks are appropriate integration work. Avoid a custom renderer solely to shrink this assessment.

Delivery with accountable ownership

Conditional pilot estimate: 4 weeks; 38 base person-days + 8 contingency = 46 person-days. Re-estimate after confirming API, file service and workflow contracts. This is separate from the assignment's suggested 10-12 candidate hours.

Work package	Base effort
Discovery, access rules, API and acceptance contract	5 person-days
React integration, conditional UI, draft/reload, languages	10 person-days
Real identity, persistence, files, workflow/status	10 person-days
Verification, UAT support, fixes, rollout and handover	8 person-days
Technical leadership, reviews and documentation	5 person-days
Contingency (approximately 21%)	8 person-days

Capacity assumption: senior frontend/solution engineer 20 days, junior frontend 10, backend engineer 8, QA 6, product/UAT owner 2. Customer test participants provide scheduled availability. No commercial price is asserted.

Four milestones

Week 1: confirm scope and prove one authenticated round trip. **Week 2:** finish request journey and files. **Week 3:** integrate workflow, audit and publication controls; verify readiness. **Week 4:** UAT, fixes, sign-off, pilot rollout and handover.

How the team is led

The solution engineer owns discovery, boundaries, trade-offs and evidence. Give the junior engineer bounded list/status/translation tasks with explicit acceptance criteria; pair on the first renderer integration. Backend engineers own server policy. QA coordinates regression. Business users own acceptance.

Use small pull requests, daily unblock sessions and two demos per week. If critical integration access is late, adjust scope or date instead of compressing testing silently.

Risks with concrete responses

Risk / owner	Response and release implication
API mismatch Solution + backend leads	Prove the contract in week 1. Missing sandbox or workflow capability triggers re-estimation.
Unsafe configuration Product/security owner	Restrict authors, component types and URLs; review scripts and preview every published version.
Unauthorized access Backend/security lead	Replace mock identity, scope queries and run negative access tests. A policy breach blocks release.
Schema/server drift Backend lead	Pin versions; test active fields and historical data. Do not publish incompatible definitions.
File risk Storage/security owner	Use approved upload/scanning/download service. The metadata demo is not evidence of file security.
Dependency and bundle Frontend lead	Lock versions, monitor advisories, regression-test updates and measure cold-load performance.
Late changes or UAT delay Sponsor + UAT owner	Book testers early, review scope weekly, assign severity/owners and retain contingency.

Handling a requirement change

Example: expensive equipment needs a second approver. Clarify threshold, currency, urgency and affected requests. Assess field/validation, API policy, workflow routing, status and UAT impact. An initial 2-4 person-day increment assumes the existing workflow already supports conditional routing; otherwise re-estimate.

Ask the sponsor to choose a date change, an agreed scope swap or deferral after the impact is concrete. Historical requests retain their original policy unless migration is explicitly approved.

Acceptance requires evidence

The business UAT owner approves outcomes and release. QA coordinates evidence; requesters and reviewers execute scenarios; the solution engineer and backend/security owners resolve defects and integration issues.

Core UAT scenario	Expected result
Change request type	Correct panel; no inactive data persisted
Incomplete final / incomplete draft	Final blocked; draft saved with valid supplied values
Save, reopen and submit	Same request and version; persisted data; read-only after submit
Other owner / reviewer access	No cross-owner access; reviewer cannot write
API failure and retry	Input retained; success only after API acknowledgement
Language / mobile / keyboard	Values retained; readable layout and usable focus/errors
Activate v2; reopen v1	No frontend rebuild; historical rendering preserved
Real files and approval workflow	Scan/access/retention and single workflow start verified in product sandbox

Entry and exit criteria

Enter UAT: agreed rules, stable build, sandbox identities, seeded data, file/workflow contracts and known limitations. **Exit:** all agreed critical journeys pass; no critical/high defects remain; lower-severity issues have accepted owners/dates; rollout/rollback are reviewed; named business owner signs the exact build and scope.

Record scenario, build, account/role, expected and actual outcome, evidence, defect ID, tester and date. Triage daily; fix -> QA retest -> user confirmation. Scope changes require impact review.

Status: local tests and selected browser checks have been executed. Customer UAT, full accessibility testing and production integration acceptance remain pending.

Demonstration and handover

15-minute demonstration

0-4 min: JSON-driven request, two conditions and validation. **4-8:** save/reload draft, language switch, injected API failure and retry. **8-10:** requester/reviewer permissions and existing status. **10-12:** active v2 plus historical v1. **12-15:** tests, boundaries and integration next steps.

Included in the source package

Runnable React/Express source and lockfile; setup README; JSON fixtures; automated tests; architecture diagram and sequence; assumptions/decision log; delivery estimate; risk register; UAT scenarios; verification record and demo script. No employer materials or real customer data are included.

Verification summary

11 automated service tests cover draft/final validation, both request types, hidden values, ownership, server-owned fields, revisions, version pinning and attachment metadata. TypeScript and production build passed. Selected browser flows and HTTP authorization checks were exercised. npm audit reported zero known vulnerabilities in the locked dependency tree on 9 October 2026.

AI use and candidate ownership

OpenAI Codex drafted the implementation, tests and documents and ran the listed checks. Assistant review is not independent candidate review. Candidate review/rehearsal is still pending: inspect the code, rerun the demo, modify the decisions as needed and update the disclosure truthfully before submitting.

Official sources and edition assumptions

Form.io React integration: github.com/formio/react
Renderer: help.form.io/dev/form-development/form-renderer
Form JSON: help.form.io/form-building/form-json
Auth model: help.form.io/dev/authentication-and-authorization

Checked 9 October 2026. Installed @formio/react 6.2.1 and @formio/js 5.6.1 declare MIT licensing. This solution uses renderer packages only; separately verify premium components, server offerings, deployment rights and support terms before procurement.